UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería



Taller 3

MLflow con scikit-learn

Diego Linares
Andy Fuentes
Christian Echeverria
Diederich Solis

Machine Learning Engineering

Octubre de 2026

Repositorio

`https://github.com/DiegoLinares11/MLFlow`

Contiene tres notebooks (`notebooks/`), el paquete con el pipeline y el seguimiento en MLflow (`src/taller3_pipeline/`), el archivo `MLproject` para correr el entrenamiento con `mlflow run`, los archivos de ambiente (`environment.yml` y `requirements.txt`), las capturas de la interfaz de MLflow (`capturas/`) y este reporte (`reporte/`).

Resumen

MLflow es una plataforma de código abierto para administrar el ciclo de vida de un modelo: registra cada entrenamiento (parámetros, métricas, datos y archivos), guarda los modelos en un formato estándar y lleva un registro de sus versiones. Para aprenderlo:

1. **Revisamos la documentación** y elegimos tres páginas: *Projects*, *Model Registry* y *Dataset Tracking*.
2. **Seguimos el tutorial** *Your First MLflow Model* en un notebook de Jupyter, analizamos su generador de datos sintéticos, lo corregimos y comparamos modelos.
3. **Conectamos MLflow con nuestro pipeline de scikit-learn**, el de la Actividad 9 con el dataset de la Champions League del Ejercicio 1, con registro automático y manual, Model Registry y un `MLproject`.

Resultados principales:

Resultado	**Evidencia**
El tutorial no corre tal cual con las versiones actuales	Tres cambios: pandas 3, `name=` y el formato skops de MLflow 3.17
El efecto de cosecha del generador siempre vale cero	$\sin(x) + \sin(x - \pi) = 0$; valor máximo 2.2×10^{-16}
Con la misma semilla, los datos cambian según el día en que se generan	El 30 % de las filas cambia de demanda de un día al siguiente
La división al azar del tutorial es demasiado optimista	R^2 de 0.754 al azar contra 0.294 en el tiempo
La mejor mejora es darle la fecha a un modelo que siga la tendencia	RMSE temporal de 199.6 a 77.3 (piso de ruido: 67.9)
Con una búsqueda, el autolog deja el detalle del pipeline en los runs hijos	17 parámetros en el run padre y 106 en cada hijo
La mejor familia del pipeline de Champions es la SVC	F1 macro 0.638 en validación cruzada y 0.601 en prueba
El modelo se carga por su alias	`models:/champions-resultado@campeon` predice lo mismo que el run

1. Tres páginas de la documentación

Recorrimos la documentación de MLflow para aprendizaje automático (`https://mlflow.org/docs/latest/ml/`) y nos quedamos con tres páginas que se conectan con lo que hemos hecho en el curso.

MLflow Projects

`https://mlflow.org/docs/latest/ml/projects/`

Un *Project* es una carpeta (o un repositorio de Git) con un archivo `MLproject` que declara tres cosas: el nombre del proyecto, los **entry points** con sus parámetros (con tipo y valor por defecto) y el **ambiente** en que deben correr (virtualenv, conda o Docker). Con eso, cualquiera reproduce el entrenamiento con `mlflow run` sin saber qué script llamar.

Por qué la elegimos: junta en un solo archivo dos temas que vimos por separado. Los entry points son los de la Actividad 9, donde cada etapa del pipeline era un comando, y los ambientes son los del Taller 2. Además, cada run lanzado así queda marcado con su origen. La aplicamos en el `MLproject` de este repositorio (sección 3).

Model Registry

`https://mlflow.org/docs/latest/ml/model-registry/`

El registro de modelos es un almacén central con versiones: cada vez que se registra un modelo con el mismo nombre se crea una versión nueva, ligada al run que la produjo. Los **alias** son nombres que apuntan a una versión (por ejemplo `@champion`) y se pueden mover.

Por qué la elegimos: resuelve una pregunta que siempre queda abierta en las entregas: cuál de todos los modelos entrenados es el que se usa. Si el servicio carga `models:/nombre@campeon`, cambiar de modelo o regresar al anterior es mover el alias, sin tocar el código. Es la pieza que conecta el entrenamiento con el despliegue.

Dataset Tracking

`https://mlflow.org/docs/latest/ml/dataset/`

El módulo `mlflow.data` guarda con cada run los datos que se usaron: un nombre, una huella digital (*digest*) calculada sobre el contenido, el esquema de las columnas y el origen (la ruta del archivo, una tabla, una URL).

Por qué la elegimos: complementa lo que hicimos con DVC en la Actividad 8. DVC versiona los datos; con `mlflow.data` cada run dice exactamente con qué versión se entrenó, y si dos runs tienen huellas distintas sabemos que no son comparables. En un dataset como el de la Champions, donde ya tuvimos que quitar columnas con fuga de información, saber qué datos entraron a cada modelo es importante.

Para la sección 3 usamos además la guía de integración con scikit-learn, que es la que pide el taller (ver más abajo).

2. Tutorial: MLflow desde un notebook de Jupyter

Seguimos el tutorial *Your First MLflow Model* en el notebook `01_tutorial_primer_modelo.ipynb`. En la documentación más reciente esa dirección redirige a la página general de *Getting Started*, así que lo seguimos en la versión 3.1.3 (`https://mlflow.org/docs/3.1.3/ml/getting-started/logging-first-model/`).

1. **Servidor de seguimiento.** Lo levantamos con

   ```
   mlflow server --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 8080
   ```

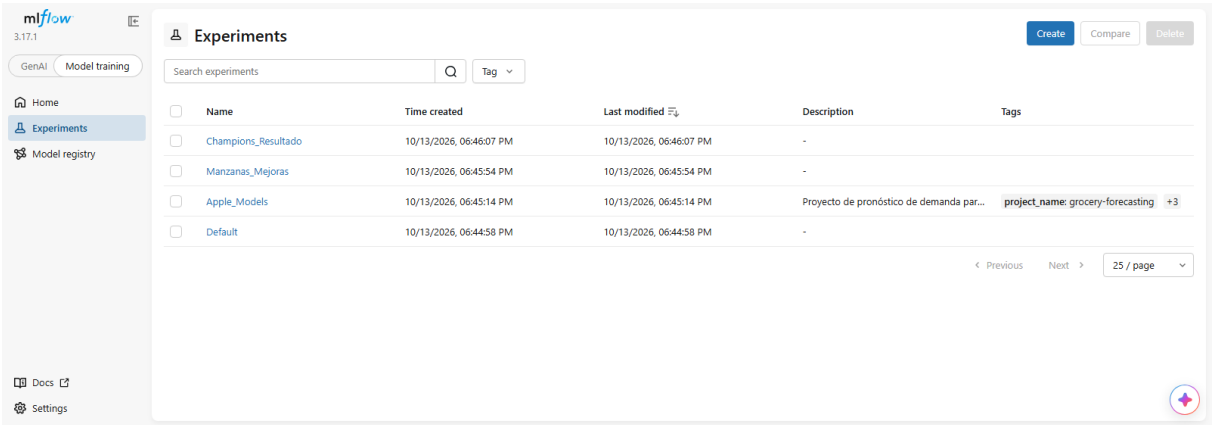
   El tutorial no indica dónde guardar; nosotros usamos SQLite para que los runs sigan ahí al reiniciar.

2. **Cliente.** `MlflowClient` lista los experimentos (al inicio solo existe `Default`).
3. **Experimento.** Creamos `Apple_Models` con tags de proyecto y una descripción.
4. **Búsqueda.** `search_experiments` con un filtro sobre el tag `project_name`.
5. **Datos sintéticos** de demanda diaria de manzanas (siguiente subsección).
6. **Primer run.** Un `RandomForestRegressor` con sus parámetros, sus métricas (MAE, MSE, RMSE y R^2) y el modelo, más tres variantes para comparar.

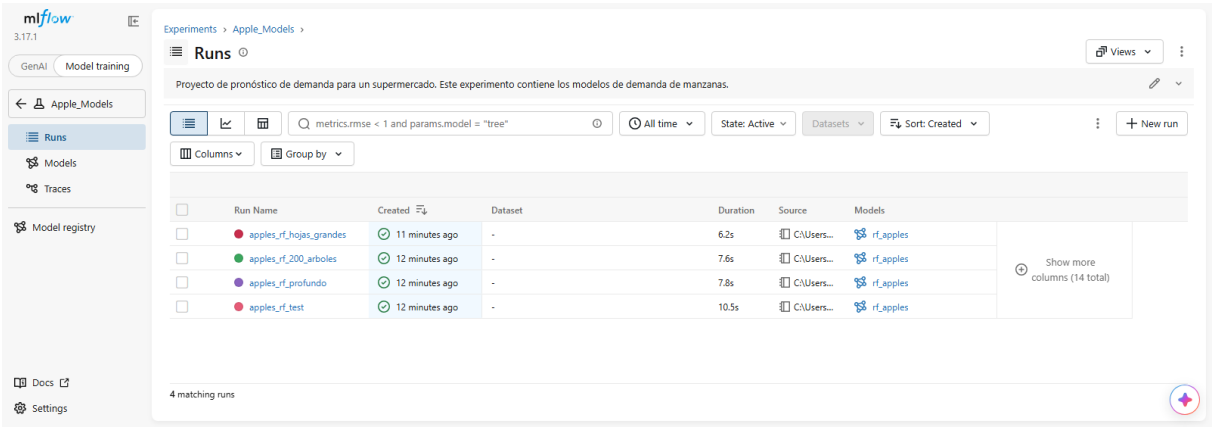
Para que corriera con las versiones actuales (MLflow 3.17.1, scikit-learn 1.9.1 y pandas 3.0.6) hubo que hacer tres cambios: `fillna(method="bfill")` ya no existe en pandas 3; en MLflow 3 el nombre del modelo se pasa con `name=` en lugar de `artifact_path=`; y en MLflow 3.17, fuera de Databricks, los modelos de scikit-learn se guardan con **skops** en lugar de pickle. Pickle puede ejecutar cualquier código al cargar un archivo; skops solo reconstruye los tipos declarados como confiables, así que el código original falla con *"untrusted types . . . sklearn.tree._tree.Tree"* hasta que se declara ese tipo con `skops_trusted_types`.

Capturas del tutorial

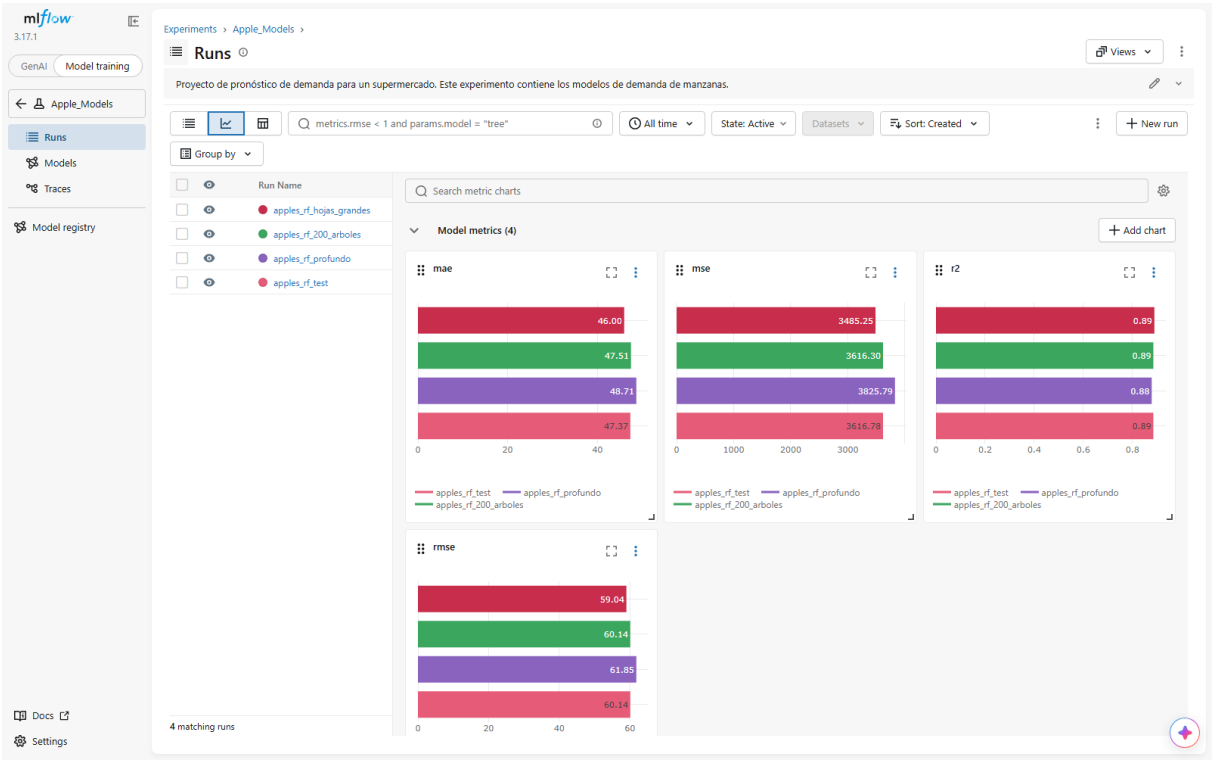
Los tres experimentos del taller en la interfaz de MLflow (modo *Model training*):



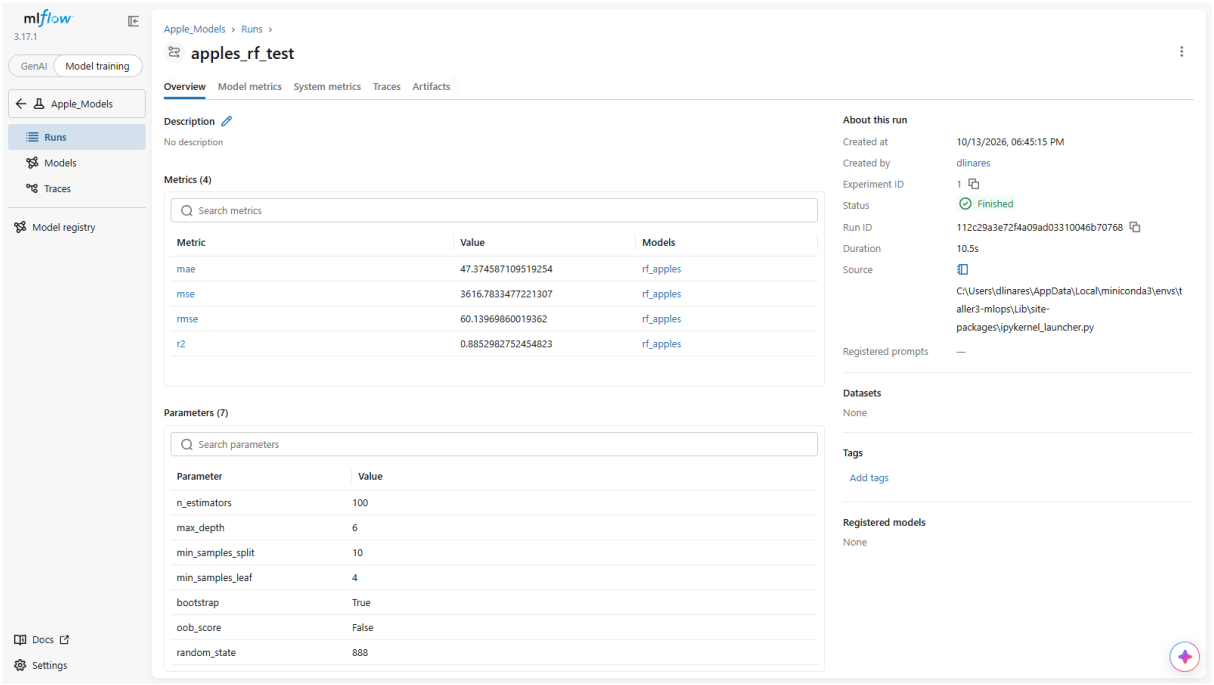
Los cuatro runs del tutorial, cada uno con su modelo (`rf_apples`):



La vista de gráficas compara las métricas de los cuatro runs. El primero (`apples_rf_test`) tiene RMSE 60.1 y R^2 0.885; la variante con hojas más grandes es la mejor por poco (RMSE 59.0) y el bosque más profundo la peor (61.9):



Detalle del primer run, con sus 4 métricas y 7 parámetros:



Análisis del generador de datos sintéticos

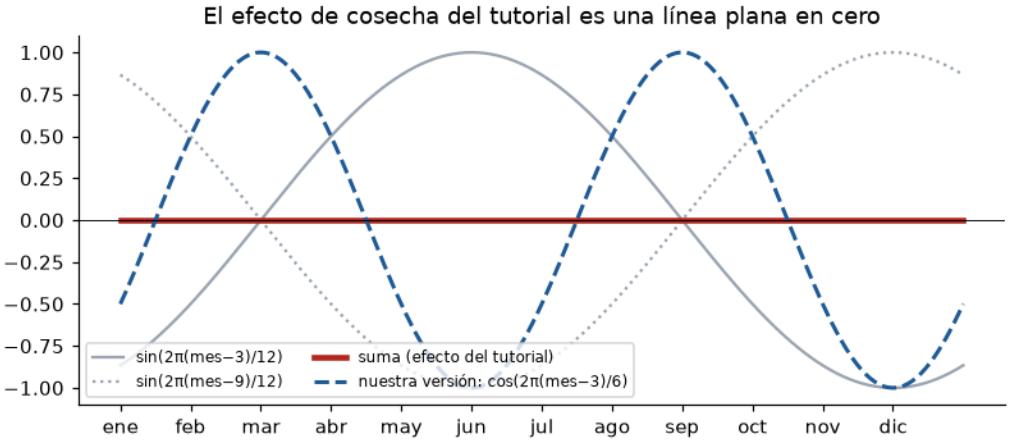
El tutorial genera la demanda con la fórmula

$$\text{demanda} = \big(1000 - 50\,\text{precio} + 50\,\text{cosecha} + 200\,\text{promo} \\ + 300\,\text{fin de semana} + \varepsilon\big) \times \text{inflación}, \qquad \varepsilon \sim N(0, 50).$$

Al revisarla en el notebook `02_datos_sinteticos_mejoras.ipynb` encontramos estos problemas, cada uno con una prueba en código:

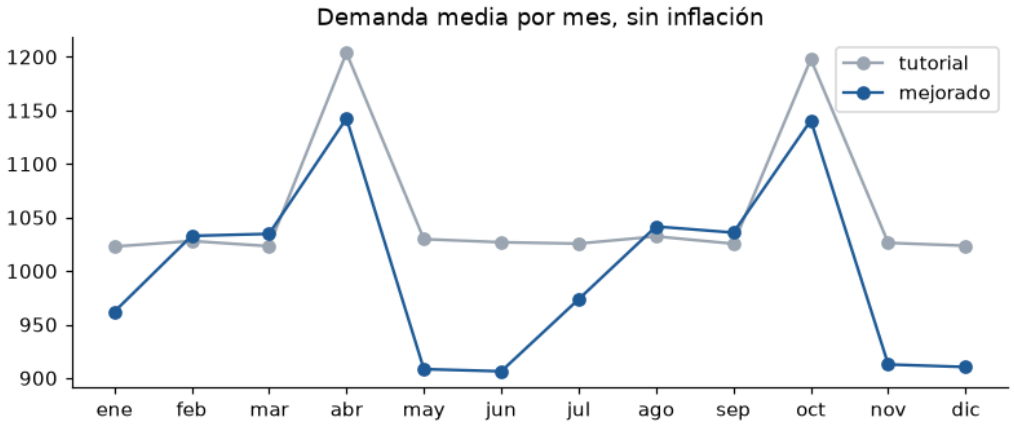
Problema	Prueba
El efecto de cosecha siempre vale cero	Suma $\sin(2\pi(m-3)/12) + \sin(2\pi(m-9)/12)$, dos senos desfasados medio periodo; el valor máximo es 2.2×10^{-16}
Las fechas salen de `datetime.now()`	Con la misma semilla, generarlo un día después cambia la demanda del 30 % de las filas (cambian los fines de semana)
Temperatura, lluvia y feriados no influyen	Su correlación con la demanda está entre -0.011 y -0.006
La inflación salta cada 1 de enero	Se calcula con el año calendario, no con el tiempo transcurrido
No corre con pandas 3	`TypeError` en `fillna(method="bfill")`
La primera fila copia su propia demanda	`previous_days_demand` de la fila 0 es la demanda de ese mismo día (fuga de información)
Ciclos de Python fila por fila	La versión vectorizada es 5.3 veces más rápida con 300 mil filas
Se evalúa con una división al azar	En una serie de tiempo hay que validar con días posteriores a los de entrenamiento

El primero es el más llamativo: el comentario del código dice que las promociones van en abril y octubre "un mes después del pico de cosecha", así que la intención eran dos picos al año (marzo y septiembre). La suma de los senos es la línea roja plana; un solo coseno de periodo seis meses, en azul, hace lo que se buscaba:



Nuestra versión, `generar_datos_mejorado`, corrige cada punto: usa ese coseno, fechas que terminan en una fecha fija, su propio generador aleatorio (`np.random.default_rng`), inflación continua, efectos para el feriado, la lluvia y la temperatura (una curva con máximo a 22 °C), descarta la primera fila y está vectorizada. Ahora dos llamadas dan exactamente los mismos datos y aparece la estacionalidad: sin la inflación, la demanda sube hacia marzo y septiembre y baja hacia junio y diciembre, además de los picos de las

promociones:



¿Se puede mejorar el modelo del tutorial?

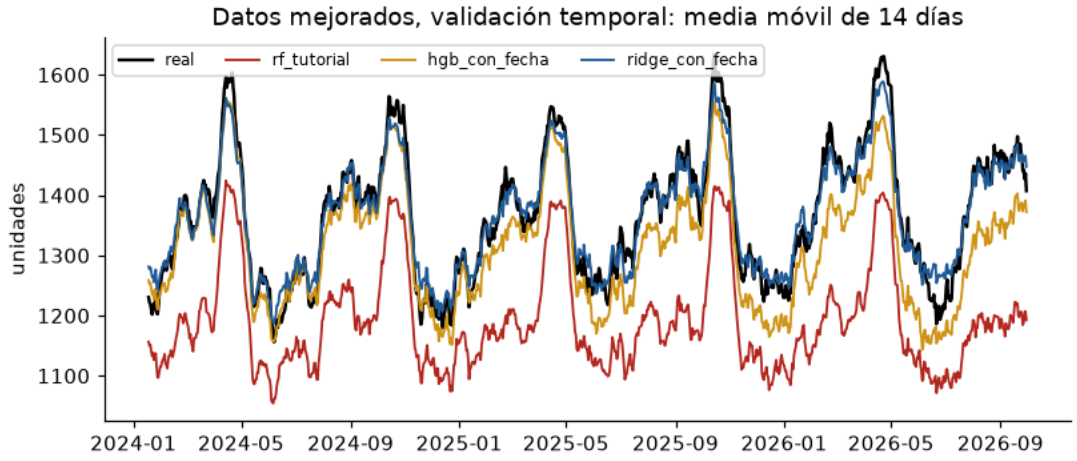
Comparamos cuatro modelos con los dos generadores y con dos formas de dividir los datos (al azar, como el tutorial, y temporal: entrenar con el 80 % inicial de los días y validar con el último 20 %). Las 16 combinaciones quedaron como runs hijos en el experimento `Manzanas_Mejoras`. Los modelos "con fecha" son pipelines de scikit-learn cuyo primer paso calcula los años transcurridos y la época del año (senos y cosenos anuales y semestrales).

	Datos del tutorial		Datos mejorados	
Modelo (RMSE)	**Al azar**	**Temporal**	**Al azar**	**Temporal**
`rf_tutorial`	124.4	199.6	127.7	205.9
`rf_con_fecha`	63.1	102.1	89.1	132.3
`hgb_con_fecha`	62.6	96.4	65.5	96.7
`ridge_con_fecha`	63.0	**77.3**	**64.4**	**79.1**
Piso de ruido	59.8	67.9	60.4	68.5

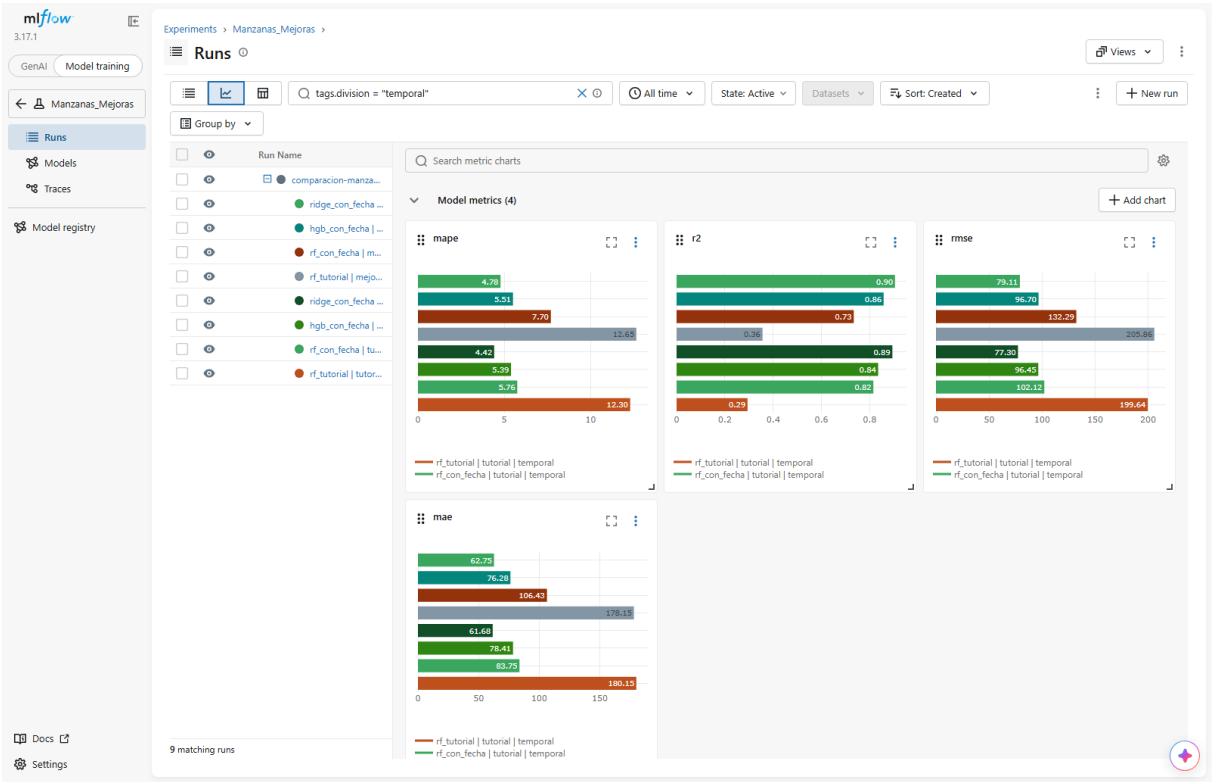
El piso de ruido es el error de un modelo perfecto: el ruido $N(0, 50)$ del generador multiplicado por la inflación, que no se puede predecir. Lo que encontramos:

- **La división al azar es demasiado optimista.** El bosque del tutorial pasa de R^2 0.754 a 0.294 cuando se valida en el tiempo.
- **La mejora más grande viene de la información.** Darle la fecha al mismo bosque baja el RMSE temporal de 199.6 a 102.1.
- **Los árboles no extrapolan la tendencia.** Por la inflación, los últimos días tienen la demanda más alta de la serie y un bosque o un boosting solo predicen valores parecidos a los que vieron. La regresión lineal sí sigue la tendencia y queda cerca del piso de ruido. Es la mejor en tres de las cuatro combinaciones; en la división al azar con los datos del tutorial el boosting le gana por 0.4.

En la división temporal sobre los datos mejorados se ve la diferencia: el bosque del tutorial (rojo) y el boosting (amarillo) quedan por debajo de la demanda real; la regresión lineal (azul) la sigue:



La vista de gráficas de MLflow, filtrada a la división temporal (`tags.division = "temporal"`), muestra los ocho runs con sus cuatro métricas:



3. MLflow con un pipeline de scikit-learn

La documentación

La sección que explica cómo usar las dos herramientas juntas es la integración de MLflow con scikit-learn (`https://mlflow.org/docs/latest/ml/traditional-ml/sklearn/`), y en su guía completa la parte *Pipeline Integration* (`https://mlflow.org/docs/3.3.0/ml/traditional-ml/sklearn/guide`). Dice que los pipelines son "ciudadanos de primera clase": con `mlflow.sklearn.autolog()`, cada `fit` registra los parámetros de `get_params(deep=True)` (los de cada paso), las métricas, el modelo con su firma y, con `GridSearchCV` o `RandomizedSearchCV`, un run hijo por cada una de las mejores

combinaciones.

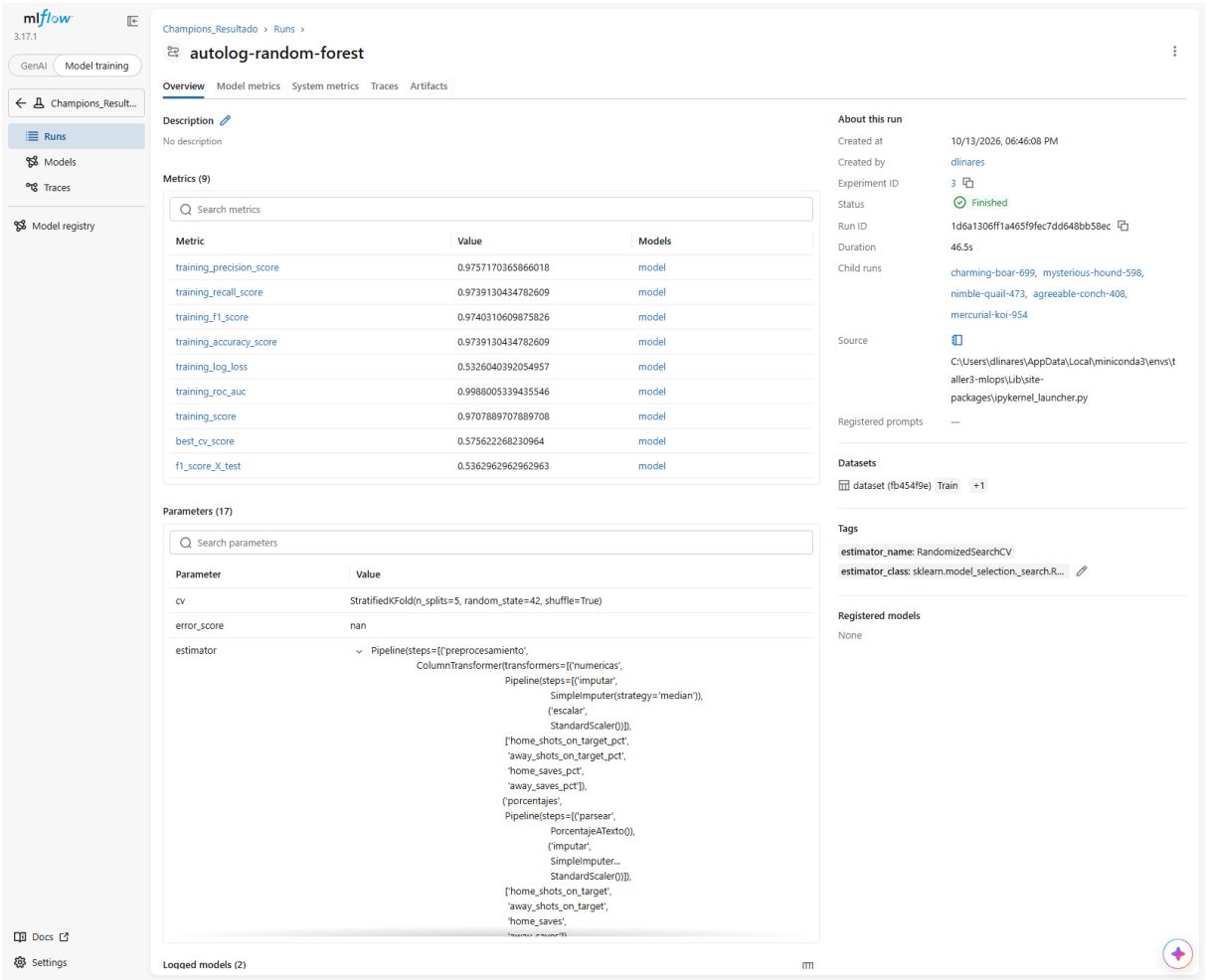
El código de ejemplo arma un `ColumnTransformer` (escalado y `SelectKBest` para las numéricas, `OneHotEncoder` para las categóricas), lo une a una regresión lineal en un `Pipeline`, activa el autolog y entrena dentro de `mlflow.start_run()`. Lo que observamos al analizarlo:

- No hay que tocar el pipeline: MLflow intercepta `fit`.
- Los parámetros se nombran con la ruta del pipeline (`preprocessor__num__scaler. ..`), la misma sintaxis `paso__parametro` que usa `GridSearchCV`.
- El ejemplo no corre tal cual: usa `X_train` sin definirlo y supone columnas que no existen.
- `OneHotEncoder(drop="first")` sin `handle_unknown` falla si en prueba aparece una categoría nueva. En nuestro dataset pasa con los equipos, por eso nuestro pipeline usa `handle_unknown="ignore"`.

Nuestro pipeline con autolog

En `03_pipeline_sklearn_mlflow.ipynb` calibramos el bosque aleatorio con `RandomizedSearchCV` (20 combinaciones, validación cruzada de 5 particiones, F1 macro) con el autolog encendido. Encontramos cuatro cosas que la guía no dice:

1. **Con una búsqueda, el run padre no tiene los parámetros del pipeline.** Tiene 17: la configuración de la búsqueda y los mejores valores. Los 106 del pipeline están en cada uno de los 5 runs hijos.
2. **Las métricas del padre son de entrenamiento** (F1 0.974), por eso se ven tan altas. La de validación cruzada es 0.576 y la de prueba 0.536.
3. **La métrica de prueba depende del orden de los imports.** El autolog reemplaza las funciones de `sklearn.metrics` al activarse; si antes se hizo `from sklearn.metrics import f1_score`, ese nombre apunta a la función original y la métrica no se registra. Nos pasó en la primera corrida.
4. **Guarda el modelo con pickle**, al contrario de `log_model`, que usa skops, y no acepta `skops_trusted_types`, así que con nuestros transformadores propios no puede usar el formato seguro.



Registro manual

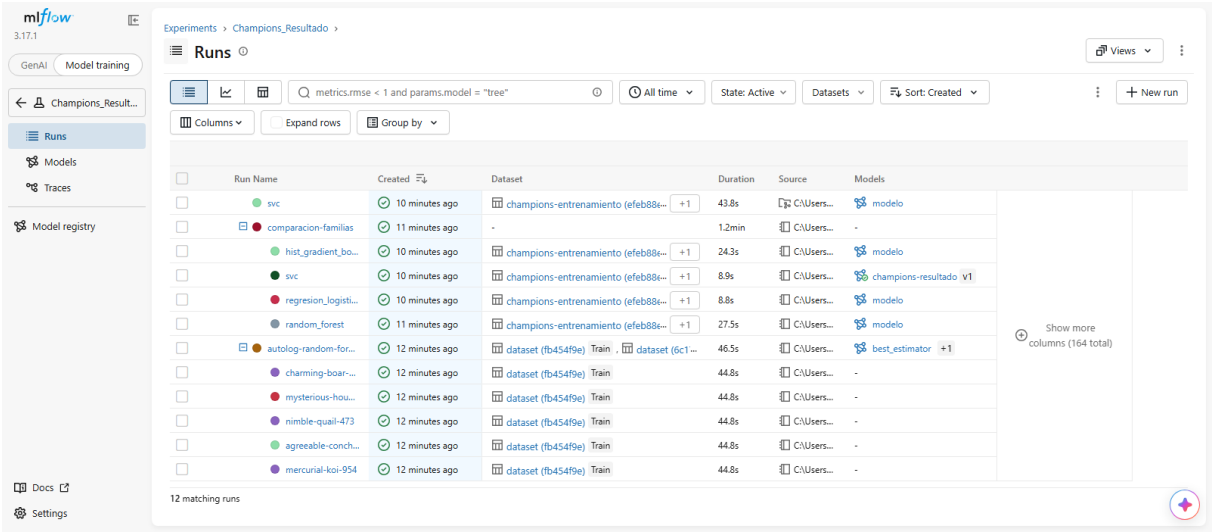
La función `entrenar_familia` (`src/taller3_pipeline/seguimiento.py`) hace la misma búsqueda, pero guarda solo lo que nos importa: los mejores hiperparámetros con nombres cortos, el F1 macro de validación cruzada, el accuracy y el F1 macro en prueba, el F1 de un modelo que siempre dice "gana el local" como referencia, los datasets de entrenamiento y prueba (con `mlflow.data`), la matriz de confusión, el reporte de clasificación y el pipeline con su firma y con el código del paquete (`code_paths`), para cargarlo en una máquina que no lo tenga instalado.

Familia	F1 macro (CV)	Accuracy prueba	F1 macro prueba
SVC	**0.638**	**0.655**	**0.601**
Regresión logística	0.611	0.621	0.542
HistGradientBoosting	0.600	0.621	0.579
Random Forest	0.583	0.621	0.576
Siempre "gana el local"			0.217

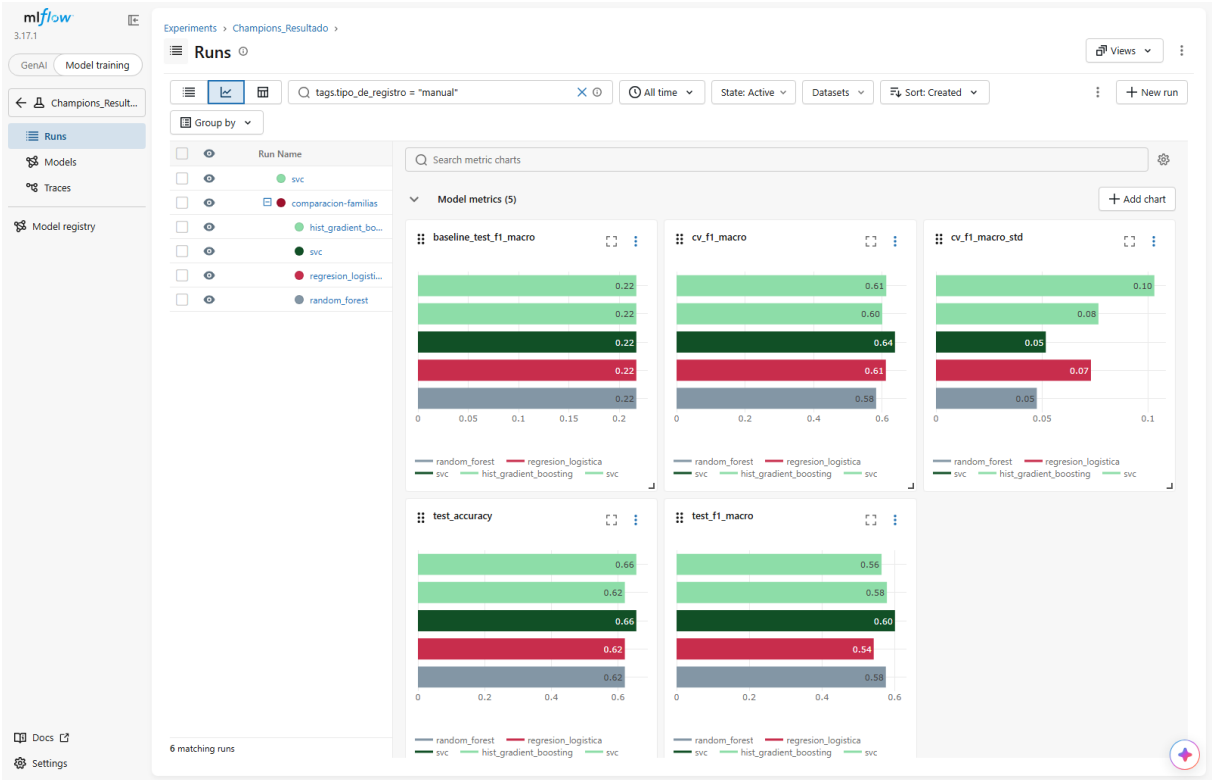
Son resultados muy parecidos a los de la Actividad 9, donde la SVC tuvo 0.652 en validación y 0.604 en prueba con 60 combinaciones por familia en lugar de 40.

Todos los runs del experimento `Champions_Resultado`: el primero (`svc`) lo lanzó `mlflow run` (su ícono de origen es distinto); debajo, la comparación de familias y la búsqueda con

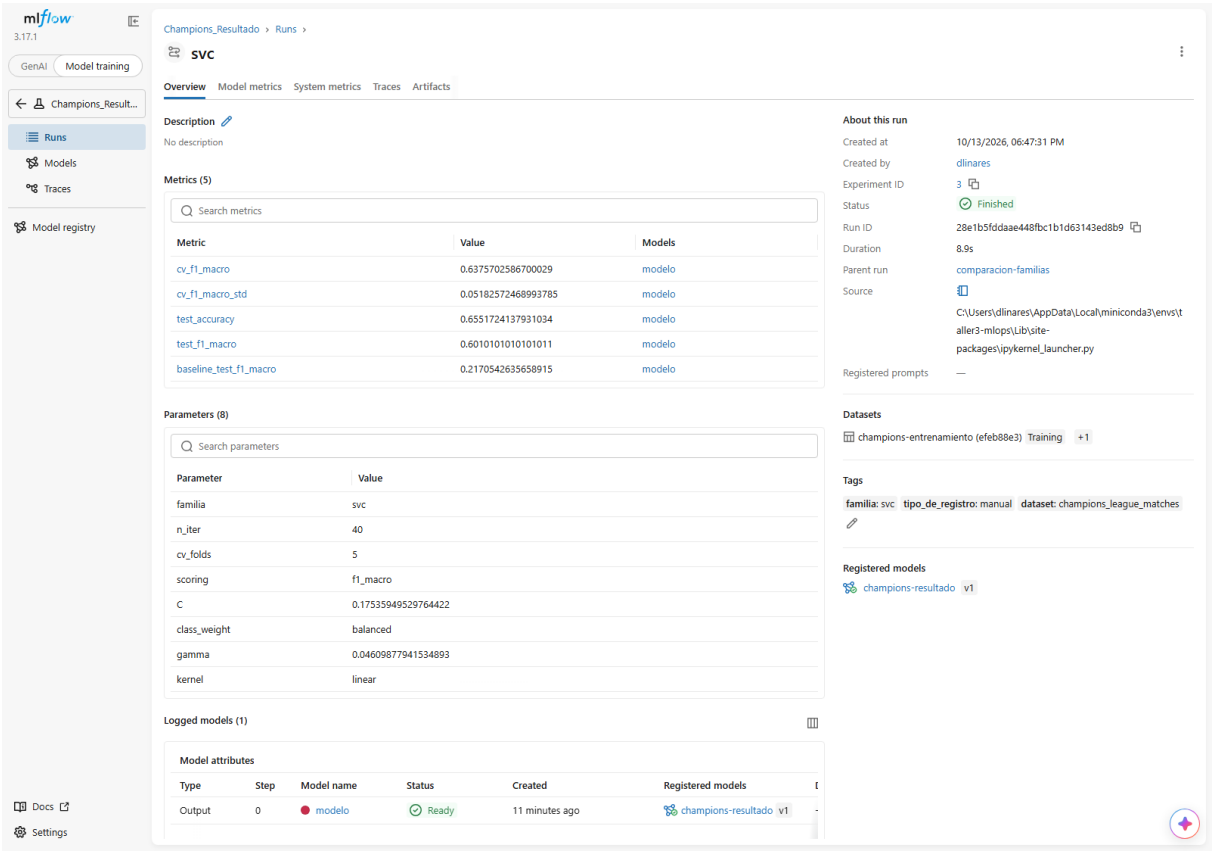
autolog, cada una con sus runs hijos:



Comparación de las cuatro familias (y el run del `MLproject`) en la vista de gráficas:



El run de la SVC con registro manual: 5 métricas, 8 parámetros, los dos datasets con su huella digital y el modelo registrado:



La matriz de confusión de prueba, guardada como archivo del run:



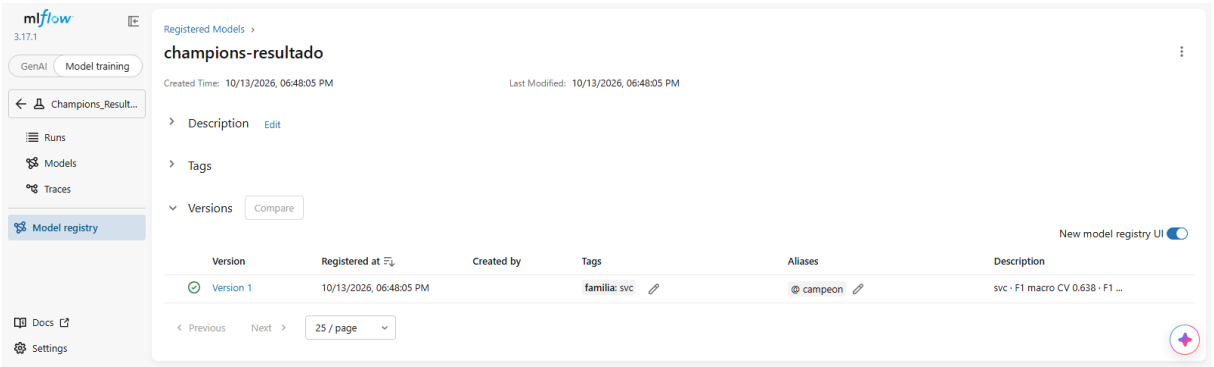
Autolog contra registro manual

	`autolog()`	**Registro manual**
Código	Una línea	Una llamada por cada cosa que se guarda
Parámetros	17 en el padre; los 106 del pipeline en cada hijo	Solo los que importan (8 en la SVC)
Métricas	De entrenamiento, más las de `sklearn.metrics` dentro del run	Las que definimos, con una referencia
Datos	Entrenamiento y evaluación como `dataset`, sin el archivo de origen	Con nombre, huella digital y archivo de origen
Gráficas	Matriz de confusión de entrenamiento	Matriz de confusión de prueba
Formato del modelo	Pickle	Skops

Nos quedamos con los dos: autolog para explorar rápido y registro manual para los runs que se van a comparar o llevar a producción.

Model Registry y MLflow Project

La SVC quedó registrada como `champions-resultado` versión 1, con el alias `@campeon`. Al cargarla por el alias (`mlflow.sklearn.load_model` con `models:/champions-resultado@campeon`) da el mismo F1 de prueba que el run (0.601) y su firma pide las 12 columnas con su tipo, incluidas las que vienen como texto (`'63%'`, `'3 of 10'`):



El `MLproject` de la raíz declara dos entry points, `main` y `registrar`, que llaman al comando `taller3-entrenar` con los parámetros `familia` y `n_iter`. Desde el notebook corrimos `mlflow run . --env-manager local -P familia=svc -P n_iter=20`, y MLflow creó el run `svc` con origen `PROJECT` (el primero de la captura del experimento).

4. Conclusiones

- **MLflow responde "¿qué probamos y con qué resultado?"** sin depender de la memoria ni de las salidas de un notebook, que se sobrescriben al volver a correrlo. En este taller quedaron más de 30 runs comparables desde la interfaz.
- **El autolog es útil, pero hay que saber leerlo:** con una búsqueda, el detalle del pipeline está en los runs hijos y las métricas del padre son de entrenamiento.
- **Para comparar en serio conviene el registro manual**, con una métrica definida, una referencia, los datos exactos y las gráficas de prueba.
- **El tutorial dejó una lección aparte:** un generador "reproducible" que no lo era y una validación al azar que hacía ver bien a un modelo que en el tiempo falla. Tener las 16 combinaciones lado a lado en MLflow fue lo que lo hizo evidente.
- **Las versiones importan:** el tutorial ya no corre tal cual, y MLflow 3.17 cambió el formato de los modelos a skops. Conviene fijar las versiones en el ambiente del proyecto.

Usos de MLflow en nuestros proyectos

- **En el curso.** Todas las actividades reentrenan el mismo pipeline de la Champions League. Con un servidor de MLflow compartido podríamos comparar entre actividades (la calibración de la Actividad 3, las familias de la Actividad 9) en lugar de copiar números entre reportes. En Databricks el servidor ya viene administrado.
- **En el trabajo.** Un modelo que se reentrena cada semana, como unas alertas de quiebre de inventario o un pronóstico de demanda, necesita saber qué versión está en uso, con qué datos se entrenó y si la nueva es mejor antes de cambiarla. Eso es el Model Registry con alias más `mlflow.data`; con un `MLproject`, otra persona puede reentrenarlo con un solo comando.
- **Con DVC.** DVC versiona los datos y MLflow los experimentos y los modelos; juntos dicen qué datos y qué parámetros produjeron cada versión.
- **En proyectos personales.** Cualquier notebook de experimentación, como los de Minería de Datos, gana con una línea de `autolog`: no se pierde qué hiperparámetros dieron cada resultado.

Referencias

MLflow. (s.f.-a). *MLflow Dataset Tracking*. Documentación de MLflow. `https://mlflow.org/docs/latest/ml/dataset/`

MLflow. (s.f.-b). *MLflow Model Registry*. Documentación de MLflow. `https://mlflow.org/docs/latest/ml/model-registry/`

MLflow. (s.f.-c). *MLflow Projects*. Documentación de MLflow. `https://mlflow.org/docs/latest/ml/projects/`

MLflow. (s.f.-d). *MLflow Scikit-learn Integration*. Documentación de MLflow. `https://mlflow.org/docs/latest/ml/traditional-ml/sklearn/`

MLflow. (s.f.-e). *Scikit-learn with MLflow* (versión 3.3.0). Documentación de MLflow. https://mlflow.org/docs/3.3.0/ml/traditional-ml/sklearn/guide

MLflow. (s.f.-f). *Your First MLflow Model: Complete Tutorial* (versión 3.1.3). Documentación de MLflow. https://mlflow.org/docs/3.1.3/ml/getting-started/logging-first-model/

scikit-learn developers. (s.f.). *Model persistence*. Documentación de scikit-learn. https://scikit-learn.org/stable/model_persistence.html